

API Sentinel

MASTER BUILD SPECIFICATION

AI-Executable Technical Blueprint for a Complete Java-Based Prototype

Version: 2.0 **Primary Language:** Java 21 **Backend:** Spring Boot **Database:** PostgreSQL **Cache:** Redis

Purpose of this document

This document is the single source of truth for implementing API Sentinel. An AI coding agent must be able to use it to create the complete repository, backend, frontend, database, tests, Docker environment and documentation without inventing missing requirements. Where implementation choices remain open, the agent must prefer the specified stack and the simplest production-sensible solution.

Product statement

API Sentinel is an API governance gateway that protects applications from excessive traffic, API abuse, unexpected cost, provider failures and operational blind spots. It authenticates clients, applies rate limits, quotas and budgets, optionally serves safe cached responses, forwards requests to configured upstream APIs, records usage and cost, detects anomalies and exposes an operational dashboard.

Non-negotiable implementation rule

The final repository must be runnable from a clean checkout using documented commands. No core feature may be represented only by a screenshot, hardcoded dashboard number, fake success response or TODO.

1. Product Definition

1.1 Problem

External APIs create operational and financial risk. A runaway loop, leaked API key, abusive client, unexpected traffic spike or failing upstream can increase request volume, cost and latency.

1.2 Solution

Place API Sentinel between consuming applications and configured upstream APIs. Every protected request passes through authentication and policy checks before reaching the upstream service.

1.3 Primary value

Prevent problems before they become incidents: control request velocity, total consumption and spend; isolate unhealthy providers; reduce duplicate calls through caching; and provide evidence through logs and analytics.

1.4 Prototype principle

Build a complete vertical slice. The prototype must be functional end-to-end, but avoid unnecessary distributed complexity that is not required to prove the product.

2. Goals and Non-Goals

Goals

Working authentication, application management, API registration, API keys, gateway proxy, rate limiting, quotas, spending caps, usage tracking, configurable pricing, Redis caching, timeout handling, bounded retries, circuit breaker, anomaly rules, dashboard, logs, health checks and automated tests.

Non-goals

Do not initially build Kubernetes, multi-region deployment, a full payment processor, enterprise SSO, a machine-learning anomaly model, a generalized service mesh or hundreds of provider-specific integrations.

3. Technology Contract

Backend

Java 21; Spring Boot 3.x; Spring Web; Spring Security; Spring Data JPA; Hibernate; Bean Validation; Actuator; Maven.

Database

PostgreSQL. Use migrations with Flyway. Do not rely on Hibernate auto-creation for production schema.

Cache

Redis. Use it for cache entries and distributed/shared rate-limit state when enabled.

Gateway client

Use Java's Spring-supported HTTP client stack such as WebClient or RestClient. The implementation must support configurable timeouts and controlled retries.

Rate limiting

Bucket4j or an equivalent token-bucket implementation. Use Redis-backed state when shared state is required.

Resilience

Resilience4j for retry, timeout and circuit breaker policies.

API documentation

OpenAPI/Swagger.

Testing

JUnit 5, Mockito, Spring Boot Test, Testcontainers.

Frontend

React + TypeScript + Vite + Tailwind CSS. Use a chart library such as Recharts.

Infrastructure

Docker and Docker Compose for local PostgreSQL and Redis. Provide a reproducible development environment.

Logging

SLF4J + Logback with structured, request-correlated logs where practical.

4. Architecture

4.1 Logical architecture

Browser → React frontend → Spring Boot REST API. Protected external requests enter the Spring Boot gateway. The gateway consults security and policy services, Redis where required, PostgreSQL for durable configuration/usage data, and then calls the configured upstream API.

4.2 Layering

Controllers handle HTTP concerns only. Services contain business rules. Repositories handle persistence. DTOs cross API boundaries. Entities represent persistence. Gateway/client components handle upstream HTTP. Policy components independently evaluate authentication, rate limit, quota and budget.

4.3 Core request pipeline

Receive request → assign correlation/request ID → authenticate API key → load API configuration → validate API status → rate-limit decision → quota decision → budget decision → cache lookup if enabled → upstream request if cache miss → timeout/retry/circuit-breaker handling → usage/cost recording → anomaly evaluation → response.

4.4 Separation of concerns

The gateway must not contain database queries, JWT code, cost formulas and Redis implementation details all in one class. Keep each responsibility testable.

5. Repository Structure

```
api-sentinel/ ■■■ backend/ ■■■ src/main/java/com/apisentinel/ ■■■ ApiSentinelApplication.java ■■■ auth/
■■■ application/ ■■■ api/ ■■■ apikey/ ■■■ gateway/ ■■■ ratelimit/ ■■■ quota/ ■■■
budget/ ■■■ cost/ ■■■ usage/ ■■■ anomaly/ ■■■ cache/ ■■■ resilience/ ■■■ audit/ ■■
■■■ admin/ ■■■ exception/ ■■■ config/ ■■■ common/ ■■■ src/main/resources/ ■■■
application.yml ■■■ application-dev.yml ■■■ db/migration/ ■■■ src/test/java/ ■■■ pom.xml ■■■
frontend/ ■■■ src/ ■■■ api/ ■■■ components/ ■■■ layouts/ ■■■ pages/ ■■■ hooks/ ■■■
context/ ■■■ types/ ■■■ utils/ ■■■ router/ ■■■ package.json ■■■ vite.config.ts ■■■ docker/ ■■■
docker-compose.yml ■■■ .env.example ■■■ README.md ■■■ docs/
```

6. User Roles

USER

Can manage owned applications, APIs, keys, budgets and usage data.

ADMIN

Can view platform-level health, users, applications, APIs, anomalies and audit records; can disable/revoke resources where authorized.

Authorization rule

A normal user must never access another user's application, API, key, budget or usage records by changing an ID in the URL.

7. Data Model

users

id UUID/UUID-compatible identifier; email unique; password_hash; role; status; created_at; updated_at.

applications

id; owner_id FK users; name; environment; status; created_at; updated_at.

api_configurations

id; application_id FK; name; provider; base_url; status; rate_limit_per_minute; daily_quota; monthly_quota; daily_budget; monthly_budget; timeout_ms; cache_enabled; cache_ttl_seconds; created_at; updated_at.

api_keys

id; application_id FK; key_prefix; key_hash unique; status; expires_at; last_used_at; created_at; revoked_at.

pricing_rules

id; api_id FK; pricing_type; request_price; input_unit_price; output_unit_price; unit_name; effective_from; active.

usage_records

id; request_id unique; api_id; application_id; api_key_id; status_code; latency_ms; input_units; output_units; estimated_cost; cache_hit; rejected; rejection_reason; created_at.

budgets

id; api_id; period_type; limit_amount; warning_percent; critical_percent; blocking_enabled; current_period_start; updated_at.

anomalies

id; api_id; type; severity; observed_value; expected_value; threshold; status; detected_at; resolved_at; description.

audit_logs

id; actor_user_id; action; resource_type; resource_id; metadata; created_at.

8. Database Rules

Constraints

Email, key hash and other logically unique identifiers must have unique constraints. Foreign keys must be enforced. Monetary values should use NUMERIC/BigDecimal semantics, not floating point.

Indexes

Index users.email; applications.owner_id; api_keys.key_hash; usage_records.request_id; usage_records.application_id + created_at; usage_records.api_id + created_at; anomalies.api_id + detected_at; audit_logs.actor_user_id + created_at.

Transactions

Budget updates, usage persistence and other state changes that must be atomic must execute inside appropriate database transactions. Avoid holding database transactions open while waiting on an external HTTP API.

Pagination

All potentially large list endpoints must support page/size or cursor-style pagination and bounded page sizes.

9. Authentication and Security

Registration

Validate email and password. Hash passwords with BCrypt or Argon2. Never store plaintext passwords.

Login

Issue a signed JWT after successful authentication. Keep signing secrets outside Git.

JWT

Validate signature, expiration and required claims. Do not place sensitive information in claims.

API keys

Generate high-entropy random keys. Show the full secret only at creation if possible. Store a cryptographic hash for lookup and retain only a safe prefix for display.

Provider credentials

If an upstream credential must be retrievable for forwarding, encrypt it at rest. The encryption key must be external to the database and source code.

Authorization

Enforce resource ownership and role permissions on every protected endpoint.

Input security

Validate URLs, numeric limits, names and request parameters. Reject malformed configuration.

Logging security

Never log passwords, JWTs, raw API keys, provider secrets, payment credentials or sensitive request bodies.

Browser security

Configure CORS explicitly for known frontend origins. Do not use wildcard origins in production configuration.

10. API Configuration Requirements

Create API

Allow an authenticated user to register an upstream API with name, provider, base URL, status, rate limit, quota, budget, timeout and cache settings.

Update API

Allow configuration changes with validation. Existing usage records must remain historically accurate.

Disable API

Disabled APIs reject gateway requests with a clear error and do not call the upstream provider.

Pricing

Pricing rules are versioned/effective-dated so historical usage can retain the correct cost basis.

Provider abstraction

Do not hardcode Gemini/OpenAI/etc. into the core gateway. Use a generic upstream configuration model and an adapter only where provider-specific request/response handling is needed.

11. Gateway Contract

Protected endpoint

Provide a gateway route such as POST /gateway/v1/{apiAlias}/request or an equivalent documented route. The exact route must be consistent across frontend, backend and README.

Forwarding

The gateway must construct an upstream request from validated configuration and client input. Do not permit arbitrary destination URLs supplied by untrusted clients; prevent SSRF by allowing only registered upstream destinations.

Request ID

Generate or validate a safe correlation ID and return it in the response headers.

Response

Preserve appropriate upstream status semantics while wrapping Sentinel-specific errors consistently. Do not leak internal stack traces.

Usage

Every accepted or rejected gateway request must generate enough metadata to explain what happened, subject to privacy/security restrictions.

12. Rate Limiting

Algorithm

Implement token bucket. Make capacity and refill/rate configurable per API or API key policy.

Decision

If tokens are available, consume and continue. Otherwise return 429 and Retry-After when calculable.

Identity

At minimum support API-key-level or application-level limiting. Do not trust an arbitrary client-supplied identity header.

Distributed state

Redis is the shared state store when multiple backend instances are simulated. Use atomic Redis operations or a proven library implementation.

Test requirement

Concurrent requests must not materially exceed the configured limit because of race conditions.

13. Quotas

Periods

Support daily and monthly request quotas in the prototype.

Reset

Use an unambiguous server-side period boundary, preferably UTC, and document it.

Enforcement

When a quota is exhausted, reject with 429 or a documented quota-specific status strategy. Do not call the upstream provider.

Atomicity

Quota consumption must be safe under concurrent requests.

14. Spending and Budget Engine

Budget

Support daily and monthly monetary limits per API.

Thresholds

Warning and critical percentages are configurable. Blocking at 100% is configurable but enabled by default in the demo.

Atomicity

Do not allow concurrent requests to race past the budget due to stale reads.

Precision

Use BigDecimal for monetary calculations.

Response

Budget rejection returns a machine-readable code such as BUDGET_EXCEEDED.

15. Cost Engine

Request-priced APIs

$\text{cost} = \text{request_count} \times \text{request_price}$.

Unit-priced APIs

$\text{cost} = \text{input_units} \times \text{input_unit_price} + \text{output_units} \times \text{output_unit_price}$, plus request price where configured.

Rounding

Define a fixed monetary scale and rounding mode; use BigDecimal.

Historical accuracy

Usage records must retain the calculated estimated cost so later pricing changes do not silently rewrite history.

Prototype provider

Include at least one seeded pricing configuration so the demo works without requiring a real paid provider.

16. Redis and Caching

Cache key

Use a deterministic key containing API identity and a cryptographic hash of the cacheable request parameters.

TTL

Use per-API TTL with a safe upper bound.

Cache safety

Only cache responses explicitly configured as cacheable. Never cache sensitive user-specific responses by default.

Stampede control

For the prototype, prevent obvious duplicate upstream calls where practical; do not introduce complex distributed locking unless needed.

Failure mode

Redis failure must not necessarily take the entire gateway down. Fall back to a documented degraded mode where safe.

17. Resilience

Timeout

Configure connection and response/read timeouts. Never allow an upstream call to hang indefinitely.

Retry

Retry only transient failures. Use bounded attempts and exponential backoff. Do not blindly retry 4xx errors.

429

Respect Retry-After where possible.

Circuit breaker

Use Resilience4j. Define failure threshold, sliding window, open-state wait duration and half-open permitted calls in configuration.

Fallback

Return a controlled upstream-unavailable response when the circuit is open.

18. Anomaly Detection

Approach

Use deterministic rules in V1; ML is not required.

Traffic rule

Flag when current request volume exceeds a configurable multiple of a baseline.

Error rule

Flag when 5xx percentage exceeds a configurable threshold.

Cost rule

Flag when current-period spend or short-window spend exceeds configured expectations.

Deduplication

Avoid creating hundreds of identical anomaly records for the same active condition.

Lifecycle

OPEN → ACKNOWLEDGED/ACTIVE → RESOLVED. Exact statuses must be consistent across API and UI.

19. Error Contract

Global handler

Use `@RestControllerAdvice`.

Error schema

Return `success=false`, `error.code`, `error.message`, `timestamp` and `requestId`.

Codes

Examples: `VALIDATION_ERROR`, `UNAUTHORIZED`, `FORBIDDEN`, `NOT_FOUND`, `RATE_LIMIT_EXCEEDED`, `QUOTA_EXCEEDED`, `BUDGET_EXCEEDED`, `API_DISABLED`, `UPSTREAM_TIMEOUT`, `UPSTREAM_UNAVAILABLE`, `CIRCUIT_OPEN`, `INTERNAL_ERROR`.

Rule

Messages must be useful to users but must not expose stack traces, secrets or internal infrastructure details.

20. REST API Specification

Auth

POST `/api/v1/auth/register`; POST `/api/v1/auth/login`; GET `/api/v1/auth/me`.

Applications

POST/GET `/api/v1/applications`; GET/PUT/DELETE `/api/v1/applications/{id}`.

APIs

POST/GET `/api/v1/apis`; GET/PUT/DELETE `/api/v1/apis/{id}`.

Keys

POST/GET `/api/v1/api-keys`; DELETE `/api/v1/api-keys/{id}`.

Usage

GET `/api/v1/usage`; GET `/api/v1/usage/summary`; GET `/api/v1/usage/cost`.

Anomalies

GET /api/v1/anomalies; POST /api/v1/anomalies/{id}/resolve.

Budgets

GET /api/v1/budgets; PUT /api/v1/budgets/{apid}.

Health

GET /actuator/health and appropriate readiness/liveness information.

Gateway

A documented gateway endpoint that accepts an authenticated API key and forwards only to a registered API configuration.

21. Frontend Requirements

Login/Register

Forms with validation, loading state, success/error feedback and authenticated redirect.

Dashboard

Show total requests, cost, error rate, active APIs, budget consumption, recent anomalies and traffic/cost charts.

API Management

List, create, edit, enable/disable and inspect registered APIs.

API Details

Show traffic, cost, latency, status distribution, cache hits, rate-limit utilisation and recent errors.

API Keys

Create/revoke keys. Display the full key only at creation and show safe prefixes afterwards.

Usage Logs

Search/filter by API, status, date and outcome. Paginate results.

Anomalies

Show severity, type, observed value, expected value, detection time and resolution status.

Settings

Edit rate limits, quotas, budgets, timeout and cache configuration.

404

Provide a custom 404 page with a useful navigation action.

Responsive design

No horizontal scroll. Tables must adapt on mobile. Navigation must collapse into a mobile menu.

Accessibility

Use semantic controls, labels, keyboard-accessible actions and readable contrast.

SEO

Set meaningful page titles, descriptions and favicon even though this is an authenticated SaaS prototype.

Links

Logo returns home. Email uses mailto. Phone uses tel where applicable. Footer links must not point to dead routes.

State handling

Every asynchronous operation must visibly represent loading, success, empty and error states.

22. Frontend Route Map

Public

/login, /register, /404.

Authenticated

/dashboard, /applications, /apis, /apis/:id, /api-keys, /usage, /anomalies, /settings.

Admin

/admin and child routes only when role=ADMIN.

Unknown routes

Redirect/render custom 404.

Navigation

Remove unused navigation entries. Do not expose routes that are not implemented.

23. Demo Data and Local Provider

Requirement

The project must be demonstrable without consuming paid external API credits.

Mock upstream

Provide a local mock upstream service or a built-in development provider that simulates latency, success, 429, 500 and timeout responses.

Seed account

Provide documented development-only credentials or a seed command. Never ship production credentials.

Seed APIs

Create at least one low-cost API and one deliberately unstable test API for demonstrating protection and resilience.

Traffic simulator

Provide a development-only script or endpoint that can generate controlled request bursts for the rate-limit/anomaly demo. It must be disabled or protected outside development.

24. Observability

Structured logs

Include timestamp, level, requestId, API/application identifiers where safe, status and latency.

Metrics

Expose request count, error count, latency and health information. Spring Boot Actuator is the baseline.

Audit

Record security-sensitive actions such as login failures where appropriate, key creation/revocation, API disablement, budget changes and administrative actions.

Health

Health endpoint must report required dependencies appropriately. Do not expose secrets or detailed internal topology.

25. Testing Requirements

Unit

Test cost formulas, token-bucket decisions, quota decisions, budget thresholds, anomaly rules, validation and key hashing.

Integration

Run Spring context tests against PostgreSQL and Redis using Testcontainers.

Security

Test unauthorized access, cross-user resource access, expired/revoked keys, invalid JWTs and privilege escalation attempts.

Gateway

Test successful proxying, upstream 4xx/5xx, timeout, retry, circuit open, cache hit/miss and disabled API.

Concurrency

Run simultaneous requests against rate-limit, quota and budget enforcement. Assert protection remains within defined tolerance.

Frontend

Test routing, protected routes, forms, API error rendering and mobile layout using an appropriate browser test framework if available.

Regression

Every completed feature must have an automated test or a documented reason why an automated test is not practical.

26. Production Checklist Mapping

Frontend

Remove horizontal scroll; fix broken links; mobile menu; favicon; page titles; meta descriptions; footer links; custom 404; dynamic copyright year; image compression; working buttons; success/error messages; remove placeholders and unused navigation; mobile overflow fixes; clickable logo/number/email; mobile optimisation.

Backend

Rate limiting; API limits; spending caps; error handling; loading/empty states; failed requests; timeouts; duplicate protection where relevant; DB query optimisation; indexes; pagination; file/upload limits if introduced; compression where appropriate; caching; uptime monitoring; error logging; simultaneous-user testing; backup/restore verification.

Important interpretation

Not every checklist item should be implemented as a superficial checkbox. Each item must be connected to an actual user flow or backend concern.

27. Docker and Configuration

Services

docker-compose should provide PostgreSQL, Redis and any local mock upstream needed for the prototype.

Environment

Use .env.example containing DATABASE_URL, DATABASE_USERNAME, DATABASE_PASSWORD, REDIS_URL, JWT_SECRET and any provider/encryption settings required by the chosen implementation.

Profiles

Provide development and production-oriented configuration separation.

Startup

README must contain a clean-checkout command sequence to start dependencies, run migrations, start backend and start frontend.

Health

Services should expose health checks where practical so startup failures are diagnosable.

28. Documentation Requirements

README

Explain product purpose, architecture, prerequisites, environment variables, setup, database migration, running, testing, API documentation and demo workflow.

Architecture document

Include component diagram, request lifecycle, security model and data model.

API docs

Expose OpenAPI/Swagger for REST endpoints.

Developer guide

Explain package responsibilities and how to add a new upstream provider or pricing rule.

Troubleshooting

Document common PostgreSQL, Redis, environment and port problems.

29. Implementation Sequence

Stage 1

Create monorepo, Maven Spring Boot backend, React frontend, Docker Compose, PostgreSQL and Redis.

Stage 2

Implement Flyway schema, entities, repositories and seed data.

Stage 3

Implement registration/login, JWT security, roles and resource ownership.

Stage 4

Implement applications, API configurations, pricing rules and API keys.

Stage 5

Implement gateway with a local mock upstream.

Stage 6

Implement rate limit, quota and budget policies.

Stage 7

Implement usage records and cost engine.

Stage 8

Implement Redis cache, timeout, retry and circuit breaker.

Stage 9

Implement anomaly detection, audit logging and Actuator metrics.

Stage 10

Implement React dashboard, API management, usage, anomalies and settings.

Stage 11

Implement unit/integration/security/concurrency tests.

Stage 12

Run full verification, fix defects, update README and prepare final demo.

30. Acceptance Criteria

AC-01

A new user can register and log in.

AC-02

A user cannot access another user's resources.

AC-03

A user can create an application and register an upstream API.

AC-04

A user can generate and revoke an API key.

AC-05

A valid gateway request reaches the registered local mock upstream.

AC-06

An invalid/revoked key is rejected before upstream access.

AC-07

Rate limits reject excessive traffic with 429.

AC-08

Daily/monthly quotas are enforced under normal and concurrent requests.

AC-09

Budget thresholds produce warnings and budget exhaustion can block requests.

AC-10

Usage records contain status, latency, cost, cache result and request ID.

AC-11

Cache hits avoid a duplicate upstream request when caching is enabled.

AC-12

Upstream timeouts return controlled errors.

AC-13

Repeated upstream failures open the circuit breaker.

AC-14

Traffic/error/cost anomaly rules create actionable anomaly records.

AC-15

Dashboard numbers are derived from backend data, not hardcoded.

AC-16

Large usage lists are paginated.

AC-17

All core REST endpoints have validation and consistent error responses.

AC-18

Automated tests pass from a clean checkout.

AC-19

Docker Compose starts required infrastructure.

AC-20

The complete runaway-traffic demo can be reproduced locally without paid API usage.

31. Final Demo Script

1. Setup

Start PostgreSQL, Redis, backend, frontend and mock upstream.

2. Configure

Log in, create an application, register a mock API with a 100 requests/minute rate limit and a defined monthly budget.

3. Normal traffic

Send normal requests. Show successful upstream responses and usage/cost metrics.

4. Rate-limit event

Generate a burst. Show 429 responses and rate-limit utilisation.

5. Anomaly

Continue controlled traffic until the anomaly rule triggers. Show the anomaly in the dashboard.

6. Cost protection

Continue until the configured budget threshold is reached. Show warning/critical state and eventual blocking if enabled.

7. Reliability

Configure the mock provider to fail or delay. Demonstrate timeout, bounded retry and circuit breaker behaviour.

8. Cache

Repeat an explicitly cacheable request. Demonstrate cache hit and reduced upstream calls.

9. Investigation

Open usage logs and anomaly details and trace the request using requestId.

10. Verification

Run automated tests and show that the same protections work under concurrent requests.

32. AI Coding Agent Contract

Role

Act as the principal software engineer and technical lead responsible for implementing API Sentinel from this specification.

Source of truth

This document is authoritative. Do not redesign the product, remove requirements, or invent unrelated features without an explicit reason recorded in the final report.

Implementation

Build real working functionality. Do not replace required functionality with mock UI, hardcoded metrics, fake API calls or TODO comments.

Quality

Use clean Java, meaningful names, small cohesive classes, dependency injection, DTO boundaries, validation, transactions where needed, secure defaults and automated tests.

Security

Never hardcode secrets. Never commit .env files containing secrets. Never expose stack traces or credentials.

External API safety

Never allow arbitrary user-controlled upstream URLs. Only registered API configurations may be contacted.

Verification

After implementation, compile backend, build frontend, run tests, start dependencies, verify database and Redis, exercise the gateway and execute the demo scenario.

Failure handling

If something fails, diagnose and fix the implementation rather than declaring completion with known errors.

Documentation

Update README and API documentation whenever the implementation changes.

Completion rule

Do not declare the project complete until acceptance criteria AC-01 through AC-20 have been checked and the final verification report records the result of each criterion.

33. Required Final AI Report

Repository summary

List implemented modules and important architectural decisions.

Run instructions

Give exact commands for local startup and testing.

Environment

List required variables and safe example values/placeholders.

Verification

Report build result, test count/result, integration result and demo result.

Acceptance matrix

For AC-01 through AC-20, report PASS/FAIL with evidence.

Known limitations

List only genuine remaining limitations. Do not hide incomplete features.

Next steps

Separate prototype improvements from production-scale improvements.

34. Definition of Done

API Sentinel is done when a clean checkout can be started with documented commands; a user can authenticate, create an application, register an API and key, route requests through the gateway, observe usage and cost, trigger rate limits and budget protection, experience controlled upstream failure, observe anomaly detection, inspect logs, and reproduce the complete demo without paid external API credits. Core functionality is covered by automated tests and the acceptance matrix is fully passed.

35. Engineering North Star

Request → Authenticate → Validate → Rate Limit → Quota → Budget → Cache → Upstream → Resilience → Usage → Cost → Anomaly → Observe → Respond.

Every major module in API Sentinel exists to make this lifecycle safer, cheaper, more reliable or more observable.